RaagaAI

Carnatic Music Generation Platform

Complete Project Documentation

For Software Developers New to AI/ML

Version 1.0 | May 2025

GitHub: Pariseashishbannu/raagaai

1. What Is This Project?

RaagaAI is a web application that composes original Carnatic music using AI. A user picks a raga (a melodic framework), a tala (a rhythmic cycle), and a section type, and the system produces a complete musical piece: a sequence of svaras (notes), matching Telugu lyrics, and a synthesised audio file (WAV).

Think of it as a backend API + React frontend that wraps an AI music-composition engine. Everything lives in one GitHub repository and can be run locally with Docker Compose or trained end-to-end in a Google Colab notebook.

> **Key insight for developers**
> The 'AI' part is one component (a small language model trained on musical notes). The rest of the codebase is standard software: a REST API, a React frontend, a PostgreSQL database, a task queue, and an audio synthesis pipeline.

What is Carnatic Music?

Carnatic music is a classical music system from South India. Two concepts are central:

- Svara: a musical note. There are 12 canonical svaras: Sa, Ri1, Ri2, Ga1, Ga2, Ma1, Ma2, Pa, Da1, Da2, Ni1, Ni2. Think of them like the 12 notes of the Western chromatic scale but with South Indian names.

- Raga: a set of rules that says which svaras are allowed, in what order, and with what ornaments. For example, raga Shankarabharanam uses Sa Ri2 Ga2 Ma1 Pa Da2 Ni2 Sa going up and the reverse going down.

- Tala: the rhythmic cycle (time signature). Adi tala = 8 beats, Rupaka = 6 beats, etc.

- Sections of a composition: Pallavi (opening verse), Anupallavi (second verse), Charanam (concluding verse), Chittaswaram (pure note passages).

What Does the System Actually Produce?

Input (user provides)	Output (system generates)
Raga (e.g. Shankarabharanam)	Svara sequence following raga rules
Tala (e.g. Adi - 8 beats)	Beat-aligned note pattern
Section (Pallavi / Charanam)	Telugu lyrics aligned to notes
Kala / tempo (Vilamba / Druta)	WAV audio file at 44.1 kHz

2. Tech Stack

Every technology used in the project, explained in plain terms.

Layer	Technology	Why it is here
Backend API	Python 3.12 + FastAPI	Async REST API framework; like Express.js but Python
Server	Uvicorn	ASGI server that runs FastAPI (equivalent to Node.js)
Frontend	React 19 + Vite 7	SPA; Vite replaces Create React App
HTTP client	Axios	Fetch wrapper used in the React app
Routing	React Router v7	Client-side routing
Database	PostgreSQL 16	Relational DB; stores users, jobs, compositions
ORM	SQLAlchemy 2 (async)	Python DB toolkit; like Prisma for Python
Migrations	Alembic	DB schema versioning; like Flyway / Liquibase
Task queue	Celery 5 + Redis	Background job runner; like BullMQ for Node
Broker	Redis	Message broker for Celery; also used for caching
Object store	MinIO	S3-compatible file storage for WAV checkpoints
Auth	JWT (HS256) + bcrypt	Standard token-based auth
Monitoring	Prometheus + Grafana	Metrics scraping and dashboards
AI model	PyTorch 2.x	Deep learning framework
Linting	ruff + mypy	Fast Python linter + static type checker
Testing	pytest + pytest-asyncio	Test runner

How the layers connect

```
Browser (React)
    |  HTTP / JSON via Axios + /api proxy
FastAPI (Uvicorn)
    |-- routers: /generate  /ragas  /talas  /users  /health
    |-- middleware: JWT auth, rate limiter (Redis), structlog
    |-- Celery: pushes generation job onto Redis queue
    |
Celery Worker
    |-- L1 RagaEngine  -> validates raga rules
    |-- L2 Tala        -> beat grid
    |-- L3 SvaraTransformer (AI model) -> svara sequence
    |-- L4 Sahityam    -> Telugu lyrics
    |-- L5 Synthesis   -> WAV audio
    |
PostgreSQL  MinIO  Redis (job state + rate limit)
```

3. Project Structure

Every folder and what lives inside it.

```
raagaai/
  src/                      Python backend source
    api/                    FastAPI application
      main.py               App entry point; registers routers & middleware
      config.py             Settings (env vars) via Pydantic BaseSettings
      routers/              One file per endpoint group
        generate.py         POST /generate  -> triggers Celery job
        ragas.py            GET  /ragas     -> list all ragas
        users.py            POST /register, POST /login
      schemas/              Pydantic request/response models (like DTOs)
      middleware/           auth.py  rate_limit.py  logging.py
    constraint_engine/
      raga_engine.py        Loads raga JSON, validates svara sequences
    ml/                     All AI/ML code
      model.py              SvaraTransformer neural network definition
      tokenizer.py          Converts svaras <-> integers
      carnatic_gpt.py       Weighted random walk generator (non-neural)
      data/
        scraper.py          Scrapes karnatik.com, shivkumar.org, Saraga
        dataset.py          PyTorch Dataset class for training
    synthesis/              Pitch contour -> WAV audio pipeline
    workers/                Celery task definitions
    db/                     SQLAlchemy models + repositories + migrations
    core/                   Constants (svara enums), exceptions, types
  frontend/                 React + Vite SPA
    src/pages/              Home  Explore  Generate  Login
    src/components/         Navbar  AudioPlayer  ProgressTracker  RagaCard
    src/hooks/              useAuth  useJobPolling
    src/api/                Axios client (proxied to localhost:8000)
  data/
    ragas/                  JSON files defining each raga's rules
    training/               svara_sequences.jsonl  (built by scraper)
    models/                 Saved PyTorch checkpoints (.pt files)
  scripts/                  seed_db.py  build_dataset.py  train_model.py
  tests/                    unit/  integration/  conftest.py
  infra/                    Docker  nginx  Prometheus  Grafana configs
  jyupiter_train_model.ipynb   Google Colab training notebook
```

4. AI / ML Concepts Explained for Developers

Who this chapter is for
You know Python, databases, and APIs but have never trained a neural network. This chapter explains every AI concept used in RaagaAI without assuming prior ML knowledge.

1 What Problem Are We Solving With AI?

We want the computer to produce a musically valid and expressive svara sequence for a given raga, section, and tempo.

The rule-based approach alone (L1/L2) produces structurally correct but musically dull output - it just walks up and down the scale. The AI model (L3) learns patterns from thousands of real compositions and produces sequences that sound like actual music: with characteristic phrases, ornaments, and natural flow.

Analogy: spell-check (rule-based) vs. auto-complete in Gmail (AI learns patterns from billions of emails). Both produce grammatically valid text, but only one sounds natural.

2 What is a Dataset?

A dataset is a large collection of examples that you show the AI so it can learn patterns. In RaagaAI:

Each example (called a 'sequence') is a list of svara names for one section of one composition, tagged with the raga and section name. For example:

```
{ "raga_id": "shankarabharanam", "section": "pallavi",
  "svaras": ["Sa", "Ri2", "Ga2", "Ma1", "Pa", "Da2", "Ni2", "Sa"] }
```

The dataset has three sources:
1. Synthetic: generated from raga rules (fast, always available, but artificial)
2. Web-scraped: real compositions from karnatik.com and shivkumar.org
3. Saraga / Dunya: research datasets of annotated Carnatic recordings

More data = better model. Augmentation (repeating sub-phrases, extracting windows) multiplies the dataset without needing more scraped data.

3 What is Tokenization?

Neural networks only understand numbers, not strings. Tokenization converts text (or in our case, svara names) into integers.

In RaagaAI the SvaraTokenizer maintains a vocabulary table like a database table:

Token (string)	ID (integer)	Meaning
<pad>	0	Padding - fills empty space in fixed-length sequences
<bos>	1	Begin-of-sequence marker
<eos>	2	End-of-sequence marker
Sa	7	The note Sa (tonic)
Ri2	9	The note Ri (second variant)

raga:kalyani	42	Conditioning: which raga to compose in
pallavi	33	Conditioning: which section
vilamba	36	Conditioning: slow tempo

Before feeding a sequence to the model, the tokenizer prepends conditioning tokens:

<bos> raga:shankarabharanam tala:adi pallavi madhyama Sa Ri2 Ga2 ...

This prefix tells the model 'I want a pallavi section in Shankarabharanam at medium tempo'. The model then continues the sequence one svara at a time.

4 What is a Neural Network?

A neural network is a mathematical function with millions of adjustable parameters (numbers called 'weights'). You give it an input (integers), it does many matrix multiplications through many layers, and produces an output (probability distribution over possible next tokens).

Think of it as a very complex function:
f(input_sequence) -> probability of each possible next svara

During TRAINING these weights are adjusted until the function gets good at predicting the next svara given the previous ones. During INFERENCE (generation) the function is frozen and used to produce new sequences one token at a time.

5 What is a Transformer? (The Model Architecture)

A Transformer is the specific type of neural network used in RaagaAI (and in ChatGPT, GPT-4, etc.). It was introduced in 2017 and is the dominant architecture for sequence modelling.

The key innovation is the 'attention mechanism': each position in the sequence can look at all previous positions and decide which ones are most relevant for predicting the next token. For music this is powerful - when generating beat 5, the model can look back at beats 1-4 to maintain musical coherence.

SvaraTransformer is a 'causal decoder' transformer, meaning:
- 'Causal' = each position can only see tokens to its LEFT (no peeking at future svaras)
- 'Decoder' = it generates output token by token, autoregressively

The SvaraTransformer has these components:

Component	What it does	Analogy
Token Embedding	Converts integer IDs to dense vectors (768 numbers per token)	Like a lookup table / dictionary
RoPE	Rotary Position Embedding - tells the model WHERE each toke	Like an index on a row in a DB
GQA	Grouped Query Attention - the attention mechanism with sharec	The 'memory' of what came before
SwiGLU FFN	Feed-Forward Network with a gating mechanism - processes ea	A multi-layer perceptron
RMSNorm	Root Mean Square Normalisation - keeps numbers in a healthy	Like standardising data before ML
x12 Layers	Stack 12 of these blocks - depth = more complex patterns learn	Like 12 layers of abstraction
LM Head	Final linear layer mapping hidden state -> vocab probabilities	The output layer / softmax

6 What is Training?

Training is the process of adjusting the model's millions of weights so it gets better at predicting the next svara. It repeats the following loop thousands of times:

```
For each batch of sequences:

1. FORWARD PASS
   Feed input tokens into the model -> model outputs probabilities
   for the next token at each position

2. LOSS CALCULATION
   Compare predicted probabilities vs actual next token
   Loss = Cross-Entropy = -log(probability assigned to correct token)
   If loss=0: model is perfectly correct. If loss is high: model was wrong.

3. BACKWARD PASS (Backpropagation)
   PyTorch automatically calculates how each weight contributed to the loss
   (the 'gradient' = direction and magnitude of change needed)

4. OPTIMIZER STEP (AdamW)
   Adjusts every weight slightly in the direction that reduces loss
   Learning rate controls how big each step is

Repeat for all batches -> one 'epoch'
Repeat for N epochs until loss stops improving
```

Key training concepts used in RaagaAI:

Epoch: one complete pass over the entire training dataset.
Batch size: how many sequences to process in parallel before updating weights. Larger = more stable but needs more GPU memory.
Gradient accumulation: process 8 small batches, add up their gradients, then update once. Simulates a batch 8x larger without extra memory.
Learning rate: how big each weight update step is. Too high = unstable training. Too low = very slow.
Warmup: start with a very small learning rate for the first 5 epochs, then ramp up. Prevents early instability.
Cosine decay: gradually reduce the learning rate toward the end of training so the model fine-tunes precisely.
Early stopping: stop training if validation loss has not improved for 15 epochs (prevents wasted compute).

7 What is Validation Loss?

You split the dataset: 95% for training, 5% held out for validation. After each epoch you measure loss on the validation set (data the model has never seen during training).

- Training loss going down: model is learning
- Validation loss also going down: model is generalising (good!)
- Training loss going down but validation loss going up: 'overfitting' - model has memorised training data instead of learning patterns. Time to stop.

The best checkpoint (saved .pt file) is the one with the LOWEST validation loss.

8 What is Inference / Generation?

Once the model is trained, you use it to generate new music. Generation is autoregressive: the model generates one token at a time, and each generated token is fed back as input for the next step:

```
Input:  <bos> raga:shankarabharanam pallavi madhyama
Step 1: model predicts -> Sa (highest probability)  -> append Sa
Step 2: model predicts -> Ri2 (given ...Sa)         -> append Ri2
Step 3: model predicts -> Ga2 (given ...Sa Ri2)     -> append Ga2
...continues until <eos> token or max_new_tokens reached
Output: Sa Ri2 Ga2 Ma1 Pa Da2 Ni2 Sa Ni2 Da2 Pa ...
```

Temperature: a parameter that controls randomness. Temperature=0 always picks the most likely token (boring, repetitive). Temperature=1 samples according to the raw probabilities (natural). Temperature=2 picks more randomly (experimental, sometimes incoherent).

Top-k sampling: only consider the top-k most likely tokens at each step. The RaagaAI model uses top_k=15 by default, filtered further by the raga constraint engine (only musically valid svaras are allowed).

9 What is fp16 / Mixed Precision Training?

Normal PyTorch operations use 32-bit floating-point numbers (fp32). The T4 GPU can do fp16 (16-bit) calculations twice as fast with half the memory.

Mixed precision (AMP - Automatic Mixed Precision): the forward pass and gradients use fp16 for speed, but the optimizer keeps a full-precision fp32 copy of the weights for accurate updates. A GradScaler multiplies the loss by a large number before backward to prevent gradients from becoming too small (underflow) in fp16.

The notebook enables this with: with autocast(dtype=torch.float16): ...

5. The SvaraTransformer Model

Architecture Summary

Hyperparameter	Notebook (T4) Value	What It Means
vocab_size	~70 tokens	Total number of distinct tokens in the vocabulary
embed_dim	768	Size of each token's vector representation
num_heads	12	Parallel attention heads (each attends to different patterns)
num_kv_heads	6	GQA: fewer KV projections -> less memory, same quality
num_layers	12	Number of stacked transformer blocks
max_seq_len	512	Maximum sequence length (512 svaras per composition)
Parameters	~78 million	Total learnable numbers in the model
VRAM needed	~9 GB	At batch=64, fp16, seq=512 on T4

Forward Pass Data Flow

```
Input: [1, 33, 9, 42, ...]   <- integer token IDs, shape (batch, seq_len)
       |
Token Embedding              <- lookup table: int -> float vector (768-dim)
       |  shape: (batch, seq_len, 768)
Dropout
       |
RoPE(seq_len)               <- generate cos/sin position encodings
       |
[TransformerBlock x12]
  for each block:
    x = x + GQA(RMSNorm(x), cos, sin, causal_mask)   <- attention
    x = x + SwiGLU(RMSNorm(x))                       <- feed-forward
       |
RMSNorm (final)
       |
Linear head (768 -> vocab_size)   <- project to vocabulary scores
       |
Output: logits (batch, seq_len, vocab_size)
        loss  = CrossEntropy(logits, labels)  <- during training
```

What Makes This Architecture Modern?

- RoPE (Rotary Position Embedding): encodes positions relative to each other, not absolute. Better generalisation to sequence lengths not seen during training.
- GQA (Grouped Query Attention): query heads = 12, key/value heads = 6. The KV cache at inference is half the size of standard multi-head attention. Used in LLaMA-3 and Gemma.
- SwiGLU FFN: uses a gating mechanism (Swish activation x a gate projection) instead of standard ReLU. Better gradient flow; used in PaLM, LLaMA, Mistral.

- RMSNorm: simpler than LayerNorm (no mean subtraction), same or better quality. Used in LLaMA.
- Weight tying: the embedding table and the final output projection share the same weights, reducing parameter count with no quality loss.

6. The Data Collection Pipeline

Before training, you need a dataset. The file src/ml/data/scraper.py collects svara sequences from five sources and saves them as a JSONL file (one JSON object per line) at data/training/svara_sequences.jsonl.

Source 1: Local Raga Synthesizer (always runs)

Reads the raga definition JSON files from data/ragas/ and generates synthetic svara sequences by randomly walking the arohanam/avarohanam patterns. No network access needed.

200 sequences per raga x ~30 ragas = 6,000 base sequences. After augmentation x3 = ~18,000 sequences. Fast but artificial - the model trained only on this would produce correct but dull music.

Source 2 & 3: Web Scrapers (karnatik.com, shivkumar.org)

Both sites have free composition notation pages written by musicians. The scrapers:
1. Crawl the index pages to find composition URLs
2. Fetch each page with a 2-second rate limit (polite crawling)
3. Parse the page text to find notation lines (strings of svara abbreviations like 'S R G M P D N S')
4. Normalise notation (e.g. 'S' -> 'Sa', 'R2' -> 'Ri2')
5. Tag each sequence with the raga name and section

The robots.txt is checked before scraping shivkumar.org.

Source 4: Saraga Dataset

The Saraga Carnatic 1.5 dataset from CompMusic Research contains 249 full concert recordings with svara phrase annotations in JSON format. You download it separately (~50 GB) and point SARAGA_DIR at the extracted folder. The scraper reads the JSON metadata files and extracts svara phrases.

Source 5: CompMusic Dunya API

A research API from Universitat Pompeu Fabra that provides pitch contour data for Carnatic recordings. The DunyaFetcher:
1. Lists recordings from the Carnatic collection
2. For each recording: fetches the tonic frequency (the Sa reference pitch)
3. Fetches the pitch contour (time-series of Hz values)
4. Converts Hz values to svara names using the formula: cents = 1200 * log2(Hz / tonic_Hz), then maps cents to the nearest semitone

Requires a free API token from dunya.compmusic.upf.edu.

Augmentation

After collecting all sequences, augment_sequences() applies two transformations to multiply the dataset size without new scraping:

1. Sub-phrase repetition: repeat a random 4-svara window in place (simulates how musicians repeat phrases

in improvisation)

2. Sub-window sampling: extract two random contiguous slices of 16-48 svaras from each long sequence

Chromatic transposition is intentionally NOT used: shifting all svaras by one semitone would produce a different raga, making the raga_id label wrong.

7. The Jupyter Notebook (jyupiter_train_model.ipyn

The notebook lets you train the model end-to-end in Google Colab (free T4 GPU) without any local setup. Run cells top to bottom.

Cell	What it does	Key code
Colab Setup	Clones repo, installs deps, sets cwd	git clone; os.chdir(); pip install
Imports & Root	Detects repo root, sets up device (GPU/CPU)	_find_repo_root(); torch.device('cuda')
Install Deps	Installs requests + beautifulsoup4 for scraper	pip install -q requests beautifulsoup4
Config	Set which scrapers to run, output path	SCRAPE_KARNATIK = True; MAX_WEB_PAGES = 50
Run Scrapers	Collects data, augments, saves .jsonl	KarnatikScraper().scrape(); augment_sequences()
Load Dataset	Wraps .jsonl in PyTorch Dataset	SvaraDataset(data_path, tokenizer, max_length=512)
Preprocess	Train/val split, DataLoaders	random_split(); DataLoader(batch_size=64)
Define Model	Builds SvaraTransformer with T4 config	SvaraTransformer(**MODEL_CFG).to(device)
Compile / Setup	Optimizer, warmup scheduler, AMP scaler	AdamW; GradScaler; SequentialLR
Train	Main training loop with fp16, grad accum, tqdm	autocast(); scaler.scale(loss).backward()
Evaluate	Final val loss on the held-out split	evaluate(model, val_loader, device)
Save / Generate	Saves checkpoint, runs inference example	model.generate(prefix_tensor, max_new_tokens=32)

Understanding the Training Loop

```
for epoch in range(1, epochs + 1):
    # --- TRAINING ---
    model.train()                              # enables dropout
    optimizer.zero_grad(set_to_none=True)      # clear previous gradients

    for step, batch in enumerate(train_loader, 1):
        input_ids = batch['input_ids'].to(device)
        labels    = batch['labels'].to(device)

        with autocast(dtype=torch.float16):    # fp16 forward pass
            out  = model(input_ids, labels=labels)
            loss = out['loss'] / GRAD_ACCUM    # scale for accumulation

        scaler.scale(loss).backward()          # fp16 backward pass

        if step % GRAD_ACCUM == 0:             # update every 8 steps
            scaler.unscale_(optimizer)         # restore fp32 gradients
            clip_grad_norm_(model.parameters(), grad_clip)  # prevent exploding grads
            scaler.step(optimizer)             # update weights
            scaler.update()                    # adjust scale factor
            optimizer.zero_grad(set_to_none=True)

    # --- VALIDATION ---
    val_loss = evaluate(model, val_loader, device)
    scheduler.step()                           # update learning rate
```

```
    if val_loss < best_val_loss:
        torch.save(checkpoint, path)          # save best model
```

What the Training Output Means

Output field	Meaning	What to look for
train X.XXXX	Average loss on training set this epoch	Should decrease each epoch
val X.XXXX	Loss on held-out data (the real metric)	Should decrease, then plateau
lr X.Xe-04	Current learning rate	Starts low, rises, then decays
XX.X min	Time elapsed	T4: ~2-3 min/epoch for 18K seqs
peak VRAM X GB	Peak GPU memory this epoch	Should stay under 14 GB on T4
[saved]	New best checkpoint written to disk	More often early, rarely late

8. The 5-Stage Generation Pipeline

When a user hits POST /generate, a Celery worker runs these five stages in sequence.

Stage L1: Raga Constraint Engine

File: src/constraint_engine/raga_engine.py

Loads raga definitions from data/ragas/*.json. Each definition specifies:
- arohanam: the ascending svara sequence
- avarohanam: the descending svara sequence
- varja_svaras: forbidden svaras
- characteristic_phrases: signature melodic motifs
- gamaka_rules: which ornaments are allowed on which svaras

The RagaEngine.validate() method checks every svara in a sequence against the allowed set for that raga. It also provides get_valid_next() to bias generation toward melodically appropriate svaras. This is applied as a logit mask during inference: forbidden svaras get -infinity probability so the model can never produce them.

Stage L2: Tala (Rhythm) Grid

Generates a beat grid for the chosen tala. Adi tala (8 beats) divides into 4+2+2 with heavy beats (sam, eduppu) on specific counts. The beat grid determines how many svaras fill each beat and which beats carry rhythmic stress. This timing information is attached to the generated svara sequence.

Stage L3: SvaraTransformer (AI Model)

The trained neural network generates the svara sequence. The conditioning prefix (raga, tala, section, kala) is tokenised and fed in, then the model autoregressively samples one svara at a time. At each step:
1. The raga constraint logit mask is applied (L1 integration)
2. Temperature scaling adjusts randomness
3. Top-k filtering keeps only the 15 most likely svaras
4. One svara is sampled from the filtered distribution
5. Repeat until <eos> or max_new_tokens

This produces a raw svara sequence like: Sa Ri2 Ga2 Ma1 Pa Da2 Ni2 Sa

Stage L4: Sahityam (Telugu Lyrics)

File: src/sahityam/

Template-based lyric generation: Telugu syllable templates are aligned to the svara positions from L3. Each svara position gets one or more syllables. The system picks templates matching the section type (pallavi uses short repeating phrases, charanam uses longer verse structures). This is purely rule-based, not AI.

Stage L5: Synthesis (Audio)

File: src/synthesis/

Converts the svara sequence to a 44.1 kHz WAV file:

1. Pitch mapping: each svara is mapped to a frequency in Hz based on the chosen tonic (Sa = e.g. 261.63 Hz for C4) and the standard Carnatic 12-TET tuning
2. Vocal synthesis: generates a sung waveform for each svara with amplitude envelope (attack, sustain, release)
3. Tambura drone: adds the characteristic Carnatic drone (Sa and Pa frequencies) in the background
4. Mix and encode: combined into a mono or stereo WAV file at 44.1 kHz / 16-bit

The resulting .wav file is uploaded to MinIO object storage and the URL is returned in the API response.

9. The Backend API

Endpoints

Method	Path	Auth?	Description
POST	/api/users/register	No	Create account. Returns JWT token.
POST	/api/users/login	No	Login. Returns JWT token.
GET	/api/ragas	No	List all available ragas.
GET	/api/ragas/{id}	No	Get one raga definition with phrases.
GET	/api/talas	No	List all available talas.
POST	/api/generate	Yes	Submit a generation job. Returns job_id.
GET	/api/generate/{job_id}	Yes	Poll job status: pending/running/done/failed.
GET	/api/compositions	Yes	List user's saved compositions.
GET	/api/compositions/{id}	Yes	Get one composition with WAV URL.
GET	/api/health	No	Health check for load balancer.

Generation Request/Response

```
# POST /api/generate
{
  "raga_id":  "shankarabharanam",
  "tala_id":  "adi",
  "section":  "pallavi",
  "kala":     "madhyama",
  "max_new_tokens": 64,
  "temperature": 0.8
}

# Response (202 Accepted)
{ "job_id": "3f7a8b2e-...", "status": "pending" }

# GET /api/generate/3f7a8b2e-...  (poll until status = "done")
{
  "job_id":   "3f7a8b2e-...",
  "status":   "done",
  "svaras":   ["Sa", "Ri2", "Ga2", "Ma1", "Pa", ...],
  "lyrics":   ["sa", "ri", "ga", ...],
  "audio_url": "http://minio:9000/compositions/3f7a8b2e.wav",
  "duration_sec": 12.4
}
```

Middleware Stack

- JWT Auth middleware (src/api/middleware/auth.py): validates Bearer token on every protected route. Decodes with python-jose, injects user_id into request state.

- Rate Limit middleware (src/api/middleware/rate_limit.py): Redis sliding window counter. Default: 20 requests/minute per IP. Returns 429 if exceeded.
- Logging middleware (src/api/middleware/logging.py): structlog with correlation IDs. Every request gets a unique trace ID for log correlation.

10. Database Schema

Table	Primary Key	Key Columns	Purpose
users	UUID	email, hashed_password, created_at	Authentication
generation_jobs	UUID	user_id, status, config(JSONB), checkpoints(JSONB)	Async job tracking
compositions	UUID	user_id, raga_id, svaras(JSONB), audio_url	Saved results
ragas	string	name_english, arohanam, avarohanam(JSONB)	Raga definitions cache
talas	string	name, beats, pattern(JSONB)	Tala definitions cache

All tables include created_at and updated_at via TimestampMixin. The ragas and talas tables mirror the JSON files in data/ragas/ (seeded by scripts/seed_db.py) for fast API responses without disk reads. JSONB columns allow flexible schema for config and checkpoint data without extra tables.

Running Migrations

```
# Create a new migration after changing a model
alembic revision --autogenerate -m "add audio_duration to compositions"

# Apply migrations
alembic upgrade head

# Or via Makefile
make migrate
```

11. How to Run the Project

Option A: Train in Google Colab (Recommended for AI training)

Ste	Action
1	Open jyupiter_train_model.ipynb in Google Colab
2	Runtime -> Change runtime type -> T4 GPU
3	Secrets (key icon) -> Add GITHUB_TOKEN -> toggle notebook access ON
4	Run Cell 1 (Colab Setup) -> clones repo, installs deps
5	Run Cell 2 (Imports) -> detects repo root and GPU
6	Run Cells 3-8 (Build Dataset) -> scrapes data, saves svara_sequences.jsonl
7	Run Cells 9-10 (Load + Preprocess) -> wraps data in DataLoader
8	Run Cell 11 (Define Model) -> builds 78M param model
9	Run Cell 12 (Compile/Setup) -> optimizer, scheduler, AMP
10	Run Cell 13 (Train) -> trains for 100 epochs, saves best checkpoint
11	Run Cell 14 (Evaluate) -> prints final val loss
12	Run Cell 15 (Generate) -> generates a sample svara sequence

Option B: Local Development with Docker

```
# 1. Clone repo
git clone https://github.com/Pariseashishbannu/raagaai.git
cd raagaai

# 2. Copy and configure environment
cp .env.example .env
# Edit .env with your secrets (JWT_SECRET, DB password, etc.)

# 3. Start all infrastructure (postgres, redis, minio, nginx, monitoring)
make infra-up

# 4. Run database migrations
make migrate

# 5. Seed database with raga/tala definitions
make seed

# 6. Start the FastAPI backend
make run-api        # -> http://localhost:8000

# 7. Start the Celery worker (in another terminal)
make run-worker

# 8. Start the React frontend
cd frontend && npm install && npm run dev  # -> http://localhost:5173
```

```
# 9. Build the training dataset (optional)
make train-data     # runs scripts/build_dataset.py

# 10. Train the model (optional, needs dataset)
make train-model    # runs scripts/train_model.py
```

Option C: Full Docker Production Build

```
make full-stack
# Runs: docker compose -f docker-compose.prod.yml up -d --build
# Starts everything in containers including nginx reverse proxy
```

Environment Variables (.env)

Variable	Example Value	Purpose
DATABASE_URL	postgresql+asyncpg://user:pw@localhost/raag	PostgreSQL connection
REDIS_URL	redis://localhost:6379/0	Redis for Celery + cache
MINIO_ENDPOINT	localhost:9000	MinIO object storage
MINIO_ACCESS_KEY	minioadmin	MinIO credentials
MINIO_SECRET_KEY	minioadmin	MinIO credentials
JWT_SECRET	your-256-bit-secret-here	Token signing key
APP_ENV	development	development | production
DUNYA_TOKEN	abc123...	CompMusic API token (optional)

12. Glossary of AI / ML Terms

The core of a transformer. Each token computes how much to 'attend' to every other token in the sequence when producing its representation.

Autoregressive

Generating one token at a time, feeding each output back as input for the next step. GPT-style models are autoregressive.

Backpropagation

The algorithm that calculates gradients (how much each weight contributed to the loss) by working backward through the network using the chain rule of calculus.

Batch size

How many training examples are processed simultaneously before updating weights. Larger batches give more stable gradient estimates.

Checkpoint

A saved file (.pt) containing the model's weights at a particular training epoch. Lets you resume training or use the model for inference.

Cross-entropy loss

The standard loss function for classification / next-token prediction. Measures how surprised the model was by the correct answer: loss = -log(p_correct).

Embedding

A learned dense vector representation of a discrete token. Each integer ID maps to a row in the embedding matrix (a lookup table).

Epoch

One complete pass through the entire training dataset.

fp16 / bf16

16-bit floating-point formats. Half the memory of fp32, faster on GPUs with Tensor Cores. Used in mixed-precision training.

Gradient

The derivative of the loss with respect to a weight. Points in the direction that would increase loss; the optimizer moves in the opposite direction.

Gradient accumulation

Accumulating gradients over N mini-batches before updating weights. Simulates a batch N times larger without extra memory.

Gradient clipping

Rescaling gradients if their norm exceeds a threshold. Prevents 'exploding gradients' that destabilise training.

GQA

Grouped Query Attention. Fewer key/value projection heads than query heads, sharing KV across groups. Reduces memory without much quality loss.

Hyperparameter

A value you set before training (learning rate, num_layers, batch_size). Not learned by the model itself.

Inference

Using a trained model to generate output (as opposed to training, which updates weights).

Learning rate

How large each weight update step is. Too high: unstable. Too low: very slow convergence.

Logits

The raw un-normalised scores output by the model before softmax. Higher logit = higher probability after softmax.

Loss function

A measure of how wrong the model's predictions are. Training minimises this. Cross-entropy is used for next-token prediction.

Overfitting

When the model memorises training data instead of learning general patterns. Detected by validation loss increasing while training loss decreases.

Parameters

The learnable numbers (weights and biases) inside a neural network. A 78M-parameter model has 78 million such numbers.

RMSNorm

Root Mean Square Normalisation. Divides activations by their RMS to keep values in a healthy range. Simpler than LayerNorm.

RoPE

Rotary Position Embedding. Encodes token positions by rotating query/key vectors. Better than absolute position embeddings for long sequences.

Softmax

A function that converts a vector of logits into a probability distribution (all values 0-1, summing to 1).

SwiGLU

A gated activation function: output = Swish(gate_projection(x)) * up_projection(x). Better than ReLU for transformers.

Temperature

A divisor applied to logits before softmax during generation. Low (<1): more predictable. High (>1): more random.

Top-k sampling

At each generation step, only consider the k tokens with the highest probability. Prevents very unlikely tokens from being sampled.

Tokenization

Converting a sequence of strings (svaras, words) into integers so a neural network can process them.

Transformer

A neural network architecture based on attention mechanisms. The dominant architecture for language models (GPT, BERT, LLaMA) and this model.

Validation set

A held-out portion of data not used for training. Used to measure how well the model generalises. Best checkpoint = lowest validation loss.

Weight decay

A regularisation technique that penalises large weights (L2 regularisation). Helps prevent overfitting. AdamW implements this correctly.